A Comparison of Machine Learning Algorithms in Identifying Higgs Boson Events From the Background

2021 Summer STEM Institute

Ourania Glezakou-Elbert

Abstract

Since its detection in the Large Hadron Collider (LHC) in 2012, the Higgs boson has remained a key element in confirming the standard model (SM) of particle physics. Current research to find Higgs boson decay channels predicted by the SM requires being able to accurately identify Higgs boson or signal events from a noisy background. Through use of machine learning, signal events can be accurately classified in offline reconstruction however, there exist many classification models which might suit this purpose. This study compares three supervised classification models in their ability to accurately and efficiently identify signal events: boosted decision trees, support vector machine, and neural networks. With an accuracy of 83.88%, F1-score of 81.72% and a training time of 8.4 seconds using the simulated dataset from the 2014 ATLAS Higgs boson Machine Learning Challenge, a histogram gradient boosted decision tree was determined to be the most effective classification model for identifying Higgs boson events. The worst performing algorithm was the support vector machine with the lowest accuracy at 80.35%, F1-score of 77.12%, and a the second lowest training time of 50 minutes.

1. Introduction

1.1. Background

Theorized to exist by Robert Brout, François Englert and Peter Higgs in 1964, the Higgs boson is the particle which is responsible for giving other elementary particles mass by interacting with what is known as the Higgs field (Englert & Brout, 1964; Higgs, 1964). The boson was not proven to exist until 2012 when the ATLAS detector at CERN observed a particle of a mass of 125 GeV following the collision of the proton bundles, which provide the mechanism for creating particles within the accelerator (Aad et al., 2012; Chatrchyan et al., 2012).

Years after the discovery of the Higgs boson at the Large Hadron Collider (LHC), there still remain questions about the nature of the particle. According to the Standard Model (SM), the Higgs boson is what gives mass to other particles and with its discovery, the Standard Model was further confirmed and pieced together. However, in order to fully understand the model and its implications regarding the nature of the universe, further research on the Higgs boson needs to be pursued.

The Higgs boson itself cannot be detected directly in the LHC, but rather, the products of the decays of the bosons are detected and used to backtrack and identify Higgs boson events. The specific particles that a Higgs boson decays into are known as a decay channel. The boson is capable of decaying into boson pairs of photons and W and Z bosons. Higgs bosons are also capable of decaying into fermion pairs such as the Higgs to tau tau decay (Gritsan et al., 2016; and, 2017) (shown in Figure 1). In order to confirm the SM, researchers at CERN use the LHC to search for predicted decay channels.

However, there are several challenges to reconstructing and studying Higgs bosons. One factor is the large amount of data that is produced during a collision of which only a small fraction will represent Higgs boson events (Figure 2). Nearly 400 events are recorded per second after having passed through a two stage trigger mechanism and of those events, only two out of approximately 1,000 will be Higgs boson events. A second issue is the noisy background. The background makes it difficult to identify the small number of Higgs boson events, or signal events, as they are frequently similar to other candidate events which result from the background. Inaccurate identification of signal events thus lead to inaccurate Higgs boson reconstructions which

[*] *Research project conducted during the 2021 Summer STEM Institute.* The Summer STEM Institute (SSI) is a six-week virtual summer program where high school students learn how to design and conduct data science and other computational research projects. To view other SSI distinguished projects, please visit www.summersteminstitute.org.

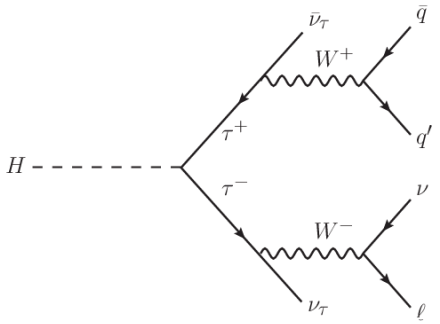


Figure 1. A Feynman diagram of a Higgs to tau decay channel (Gritsan et al., 2016)

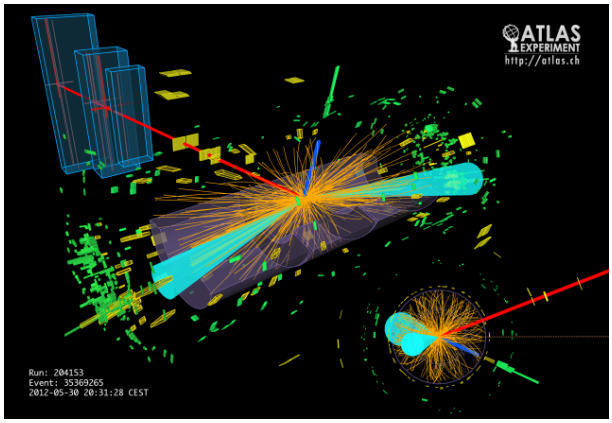


Figure 2. Visualization of a candidate Higgs boson event decaying into tau leptons. There are many jets of particles and events to account for, making it difficult to identify and reconstruct Higgs bosons accurately (Bianchi, 2013)

.

can limit the study of the particle.

As this is a complex classification problem which requires the identification of hidden correlations in a large and noisy dataset, machine learning would be an ideal approach to accurately identifying signal events.

1.2. Literature Review

1.2.1. Machine Learning in Particle Physics

Because of the large scale and complexity of the data collected from CERN –numerous features describing events, low occurrences of signal events, and quantum uncertainty which can affect the quality of the data collected–particle physicists require complex sorting and regression mechanisms in order to identify and study events, such as the Higgs boson.

Currently, it is standard for particle physicists to employ boosted decision trees for event identification and data analysis in offline reconstruction (Guest et al., 2018). However,

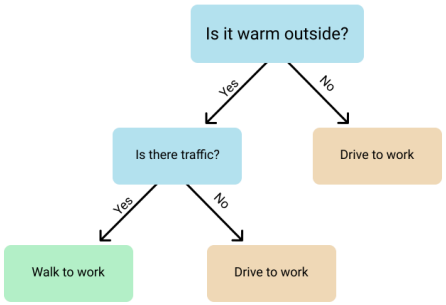


Figure 3. Representation of a DT deciding whether or not one should walk to work or drive

alternative algorithms and approaches to this data analysis have been used: support vector machines, neural networks, and deep learning, among others (Guest et al., 2018). However, which algorithm might perform best in this scenario in order to most efficiently and accurately determine Higgs boson events remains a question. This paper seeks to determine which of the following three algorithms, boosted decision trees, support vector machines, and neural networks, is most effective at identifying signal events from the noisy background.

Notably, in 2014, ATLAS held a machine learning challenge to invite scientists to develop algorithms to sort signal events (Adam-Bourdarios et al., 2015). The participants were given simulated data with events to train and then test their algorithms. The top two submissions for the competition were a deep learning neural network that won first and a gradient boosted decision tree which came in second place.

1.2.2. Boosted Decision Trees

Decision trees (DTs) are a logical, supervised regression and classification model which separates data at nodes based on certain conditions (Sen et al., 2020). It is a "greedy" algorithm, meaning that it takes a heuristic approach to problem solving and uses probabilities to estimate the best local choice of data separation at each node. Figure 3 demonstrates simplistically how a DT takes an input and sorts it through various nodes until the final node, or "leaf," as a method of classification.

Benefits to using the DT include its ability to easily be followed, understood, and visualized. Unlike neural networks, which have hidden layers, DTs can be printed out and the decisions made at each node can be viewed. Additionally, minimal data processing is needed in order to classify the data and generally, DTs have shorter training times. However, DTs can become overly complicated with larger datasets which can cause the models to not be general enough and to overfit to the data.

DTs can also be combined in order to created a boosted decision tree (BDT). BDTs combine several trees in order to increase the accuracy of the model and to address overfitting. Two common BDT algorithms are the random forest classifier and the gradient boosted decision tree. Random forest classifiers are composed of several decision trees which take random subsets of data in a process known as bagging. The results of each tree are then combined and averaged in order to produce the final classification. Taking these random subsets of data can help to reduce overfitting by forcing the tree to consider new subsets of the data rather than just being given one training dataset. Gradient boosted decision trees work by weighting cuts which are complicated or incorrect from the previous tree in the following tree, thus addressing classification issues. Currently, CERN makes use of gradient boosted decision trees for their offline classification of Higgs events

1.2.3. Support Vector Machine

A support vector machine (SVM) is a supervised classification model which works by plotting data with n different features on an n-dimensional plane. The algorithm then finds the hyperplane which is capable of dividing the data such that data points on one side of the plane belong to one class and those on the other belong to a different class (Sen et al., 2020; Sforza & Lippi, 2013).

While SVMs require low memory usage and can accurately predict data with large numbers of features, they require lots of time to train as the distance from the hyperplane to the nearest points, known as support vectors, must be calculated in order to ensure that the distance is maximized on both sides of the hyperplane. Additionally, SVMs decrease severely in performance when given a noisy dataset, such as that of a selection of Higgs boson candidate events.

There has been work regarding the optimization of SVMs within the context of research at CERN. In Ref. (Sforza & Lippi, 2013), the SVM was optimized to reduce the noisy multi-jet background in signal-background event classification through use of a biased training dataset and through variable selection and weighting. SVMs are not as commonly used as BDTs in event classification.

1.2.4. Neural Network

Neural networks are a supervised regression and classification model which consist of multiple layers of neurons. The input layer takes in data and transforms it by weighting the various features of the data, performing something comparable to a miniature linear regression. The values are summed and if the sum is above a certain threshold, it is passed to the next node until the data reaches the last layer of neurons and is classified.

While the neural network is capable of dealing with complex and noisy data with a high accuracy, the hidden layers which exist between the input and output layer are blackboxes, making the algorithm difficult to track, conceptualize, and understand. Neural networks are also resource intensive, requiring large amounts of memory and training time.

In terms of particle physics applications, deep learning, a type of neural network that is complex and involves many hidden layers, is commonly used for event reconstruction after the signal events have been identified.

2. Purpose

The purpose of this work is to answer the following research questions:

1. How can a decision tree algorithm be used to identify Higgs Boson events from the background?
2. How can a support vector machine algorithm be used to identify Higgs Boson events from the background?
3. How can a neural network be used to identify Higgs Boson events from the background?
4. How do the aforementioned three algorithms compare in terms of accuracy of identification, computational resource usage, and the time required to train and test?

3. Methodology

3.1. Data Acquisition

The training dataset provided on Kaggle by ATLAS for their 2014 Higgs Boson Machine Learning Competition will be utilized in order to train the algorithms. The dataset consists of 250,000 simulated events with 30 features which describe the simulated candidate events. Additionally, the dataset includes a weight and a label column which denotes each event as a signal or background event. Though it will not be used to train the data, the weight column helps account for the fact that there is an increased occurrence of signal events in the simulation. The simulation has an increase in the number of signal events as in the LHC, only approximately two signal events occur for every 1,000 events which would be inefficient for training the algorithms.

3.2. Data Pre-processing

Before using the dataset, it is be split into three sections for training, validation, and testing. 80% of the original dataset will be used for the training set, the remaining 20% will be evenly split between the validation and test sets.

In order to reduce the training time of the algorithms, principal component analysis (PCA) will be done. PCA is a

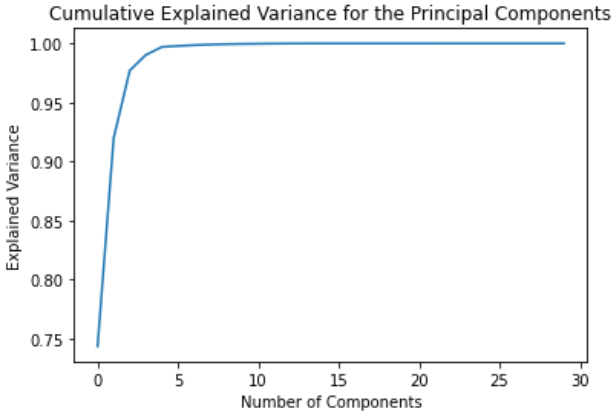


Figure 4. A graph showing the cumulative percentage of explained variance for each number of principal components. The explained variance reaches 99.9% by 8 principal components, allowing for the dataset to be reduced from 30 features to 8 as a higher percentage of explained variance above 8 components is unnecessary.

type of dimensionality reduction which reduces the number of features within a dataset by identifying the components which have the greatest amount of information or variance, thus decreasing the number of features in the dataset. The resulting values of the analysis are a linear combination of the original values. As seen in Figure 4, 99.9% of variance can be contained within 8 features. Even with only three principal components, one can observe a relatively clear separation of background and signal events, seen in Figure 5.

However, there is still some overlap between the signal and background events with just three features and thus, the first eight principal components will be utilized to train and test the neural network and support vector machine. Additionally, these two algorithms are the most resource-intensive and experience the greatest improvement in performance with a reduced training set. The difference in the performance of the boosted decision trees with the principal components is negligible and thus, these algorithms will be utilizing the raw data. To further reduce the training time for the SVM, the data which results from the PCA will also be scaled in order to reduce the distance from each support vector to the hyperplane, which in turn reduces the time it take the SVM to measure the distance from the hyperplane to each support vector.

3.3. Machine Learning Algorithms

Throughout this study, three machine learning algorithms for classification are used and compared: BDT, SVM, and neural network. In order to implement these algorithms, the machine learning library, sci-kit learn (Pedregosa et al., 2011), and the Python notebook, Google Colab, will be used.

Each algorithm requires hyperparameter tuning. Any hyperparamters not mentioned are assigned their default sci-kit learn values.

3.3.1. Boosted Decision Tree

This study makes use of three different boosted tree methods. Ensemble methods make use of multiple decision trees in order to classify data. The three boosted decision trees we are using are a random tree classifier, a gradient boosted decision tree, and a histogram gradient boosted classifier.

In order to first become familiar with the mechanics of a decision tree, a single, non-boosted tree was created, shown in Figure 6. The process of creating the tree involved tuning three hyperparameters: `max_depth`, `min_samples_leaf`, and `min_impurity_split`. The first hyperparameter controls the maximum depth of the tree, effectively how many layers it has. The second parameter controls the minimum number of samples that can be found at all leaves of the tree. The final hyperparameter controls when a node stops splitting. If the impurity, number and division of different classes, of the node is above the threshold value, then it will be split, otherwise, it will be a leaf. Tuning these hyperparameters involved looping through an appropriate range of values for each hyperparameter. In order to compare the performance of each hyperparameter value, a dictionary was created to store the hyperparameter value and the accuracy for the model when used with the validation test set. Then, a second loop was used to find the hyperparameter value with the highest accuracy. Final hyperparameter values for this tree were 9 as the `max_depth`, 11 for `min_samples_leaf`, and 0.005 for `min_impurity_split`.

Once familiarity with the model was gained, ensemble methods were implemented. Three boosted decision trees were used, the first of which was a random forest classifier. First, the tree was run without any tuning in order to see what the accuracy of the model was. After determining that the untuned tree had a similar accuracy to the non-boosted tree, tuning commenced. A hyperparameter tuning process similar to the one previously described with the non-boosted decision tree was implemented, though the hyperarpameters altered were different. For the random forest classifier, the hyperparameter that was tuned was `n_estimators`, the number of trees in the random classifier forest. The tuning resulted in a final 72 estimators in the forest.

The second boosted decision tree used was the gradient boosted decision tree. As with the random forest classifier, before any tuning was done, an untuned model was run. Unlike the random forest classifier, which had an accu-

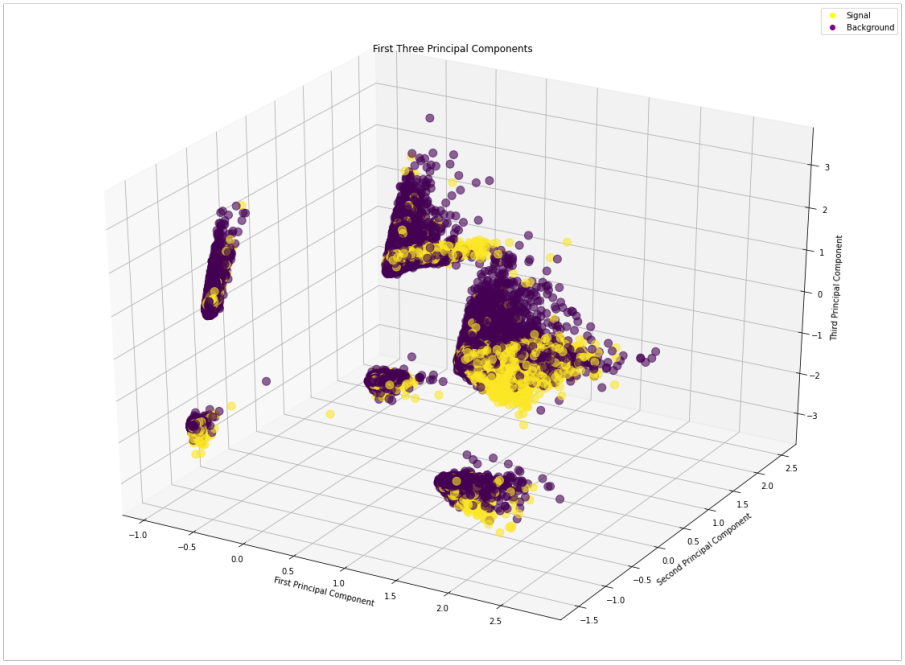


Figure 5. A graph plotting the first three principal components of the data against each other. Yellow points represent signal events while purple points represent background events. There is already a relatively clear distinction between the two in some of the clusters shown. The accuracy with which the two are classified would likely increase with a greater number of features to distinguish the two types of events from each other.

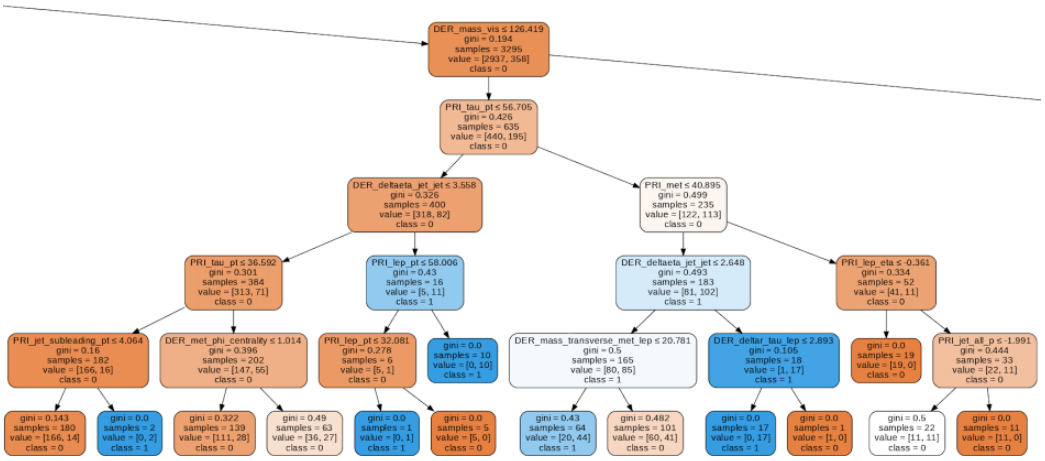


Figure 6. Part of the initial decision tree which was created in order to become familiar with the mechanisms of the decision tree. This section shows the nodes which come from the second node directly to the right of the initial node. The tree had a depth of nine which was the max depth with the highest accuracy of sorting.

racy around the 73% of the untuned non-boosted decision tree, this model had an accuracy of 54%, a significant drop. Additionally, the model took longer time to run than the previous two decision trees. Due to time constraints, this second model was dropped and not tuned in favor of examining the third boosted decision tree algorithm, a histogram gradient boosted decision tree. While both algorithms are gradient boosted, the histogram gradient boosted decision tree bins the data, allowing it to perform at a quicker rate than the other algorithm. When the untuned histogram gradient boosted tree was run, it took a significantly less of time to run and also had an accuracy comparable to that of the random forest classifier and the non-boosted decision tree. Thus, hyperparameter could be done in order to improve the accuracy of the model and reduce overfitting of the training data.

For the histogram gradient boosted decision tree, the hyperparameters tuned were `max_bins`, `max_leaf_nodes`, and `max_depth`. `Max_bins` controls the maximum number of bins in the histogram, `max_leaf_nodes` controls the maximum number of final nodes or leaves which can exist in a given tree, and as described above, `max_depth` controls the maximum number of layers in a tree. As seen in Figure 7, the F1-score and the accuracy level out around 25 bins as the maximum. Figure 8 shows the F1-score and accuracy of the model as the maximum number of leaf nodes increases. Here, the graph flattens near 30 nodes and reaches its maximum between 30 and 40 nodes. The final hyperparameter values chosen for the model were 25 for `max_bins`, 35 for `max_leaf_nodes`, and 21 for `max_depth`.

3.3.2. Support Vector Machine

Before performing any hyperparameter tuning, the SVM was trained on the training data and tested using the validation set without any pre-processing, e.g. PCA or scaling. This resulted in the SVM taking several hours to train and test which was inefficient and would pose problems when it came to hyperparameter tuning. In order to combat this, PCA was used to reduce the number of features of the data down from 30 to the 8 principal components for training and testing, and down to 4 for the hyperparameter tuning process.

The hyperparameters that were tuned for the SVM were the kernel and gamma. The kernel is the shape of the function that separates the hyperplane in order to categorize the events. The kernels tested were linear, polynomial, and radial basis function (RBF). Each kernel was used and the accuracy was recorded. The highest accuracy came from the RBF kernel. Once the RBF was selected as the kernel, other hyperparameters could be tuned. Gamma controls how close to the points the function is fit. Similar to how the hyperparameters in the decision tree algorithms were tuned, various values of gamma were iterated through and their corresponding accuracies and F1-scores were recorded in order to determine which value of gamma provided the best performance. As seen in Figure 9, the accuracy and F1-score of the SVM model levels around 2.00; thus, this value was selected to be the value of gamma.

3.3.3. Neural Network

The final algorithm follows a similar process of running the model without any hyperparameter tuning in order to get a grasp of the model. The neural network was trained using the original data without pre-processing. This resulted in a considerably long training time that approached an hour. After this, the first 8 principal components were scaled and used to train and test the model.

Next came hyperparameter tuning. For this model, the hyperparameters which were tuned were `learning_rate`, `max_iter`, and `hidden_layer_sizes`. The hyperparameter `learning_rate` controls how the weights of the inputs change as the model progresses. In order to optimize this, the three different learning rates were passed into the model's hyperparameter and the rate with the highest accuracy was chosen. This model had an adaptive learning rate which will keep the training rate the same as the inital rate unless the training rate does not decrease by enough after two iterations. The second hyperparameter, `max_iter`, controls the maximum number of times a data point can be used. In order to tune this, values between 270 and 290 were iterated through in order to find the value which produced the highest accuracy. It was found that 288 iterations gave the highest accuracy. Finally, `hidden_layer_sizes` controls the number of hidden layers and the number of neurons within each layer. Before tuning the sizes of the hidden layers, the number of hidden layers first had to be determined. Two layers would be used in order to minimize the training while still allowing for a higher accuracy. Once the number of layers had been decided, then the sizes of the layers could be altered and optimized. It was found that 175 neurons in the first hidden layer and 90 neurons in the second hidden layer provided the highest accuracy.

3.4. Evaluating the Algorithms

In order to evaluate the algorithms, three factors are evaluated: the time required to train and test, amount of computational resources used, and the accuracy of identification. Each optimized algorithm was run 6 times using a new training and test dataset in order to measure the memory usage and time required to train and test each algorithm. The accuracies and F1-scores of each algorithm for the training and test will gathered from each of the 6 iterations then averaged and a standard deviation will be determined. The F1-score

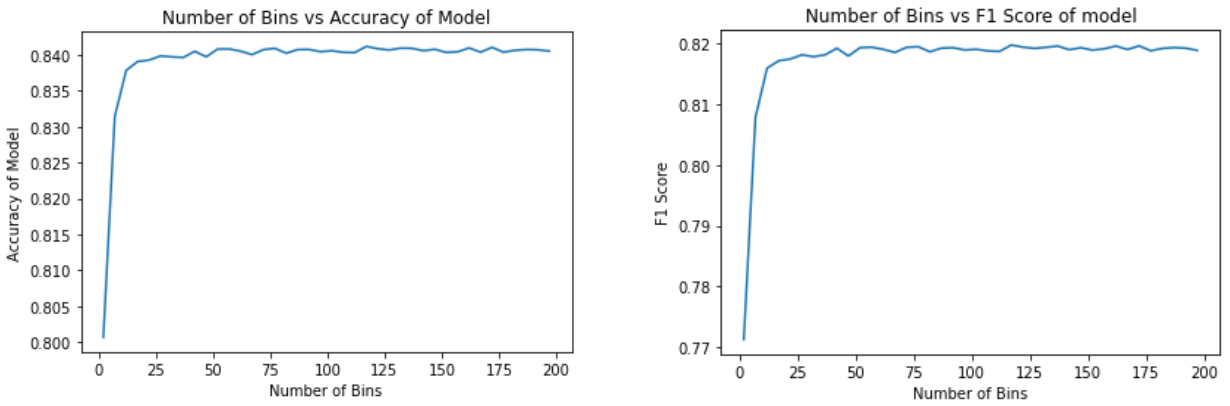


Figure 7. Left: Graph of the accuracies of the histogram gradient boosted decision tree against the maximum number of bins in the model. The accuracy levels out around 25 bins. Right: Graph of the F1-score of the histogram gradient boosted decision tree against the maximum number of bins in the model. The F1-score flattens and hits its maximum around 25 bins.

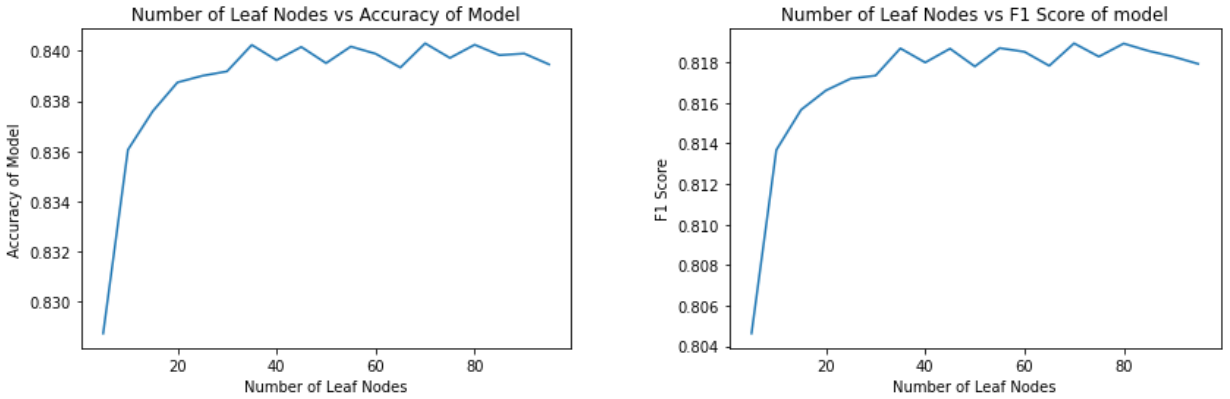


Figure 8. Left: Graph of the accuracies of the histogram gradient boosted decision tree against the maximum number of leaf nodes in a tree. Right: Graph of the F1-score of the histogram gradient boosted decision tree against the maximum number of leaf nodes in a given tree. Both graphs flatten around 30 leaf nodes and hit a maximum near 40.

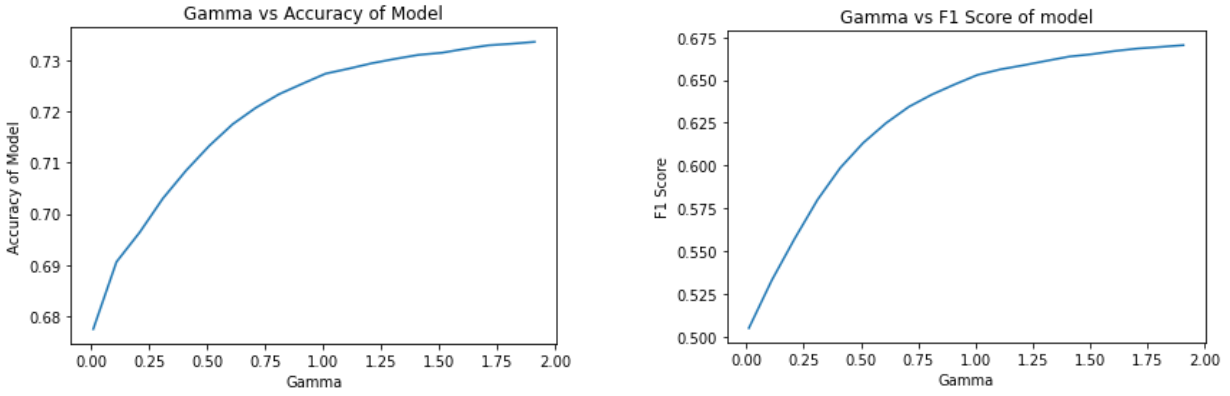


Figure 9. Left: A graph showing the accuracy of the SVM model against increasing values of gamma. Right: A graph showing the F1-scores of the SVM model against increasing values of gamma. Both graphs demonstrate an increase in the accuracy of the model with an increase in gamma however, the rate at which the accuracy increases decreases as the value of gamma increases, eventually leveling around gamma = 2.00.

is defined as

$$F_1 = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

where TP represents the true positive rate, FP represents the false positive rate, and FN represents the false negative rate. Additionally, a receiver operating characteristic (ROC) curve of each algorithm was plotted and the area under the curve determined.

4. Results and Discussion

4.1. Comparisons of the Algorithms

Table 1 shows the performance of each algorithm in the various categories which were used as points of comparison and evaluation. Overall, the BDT had the shortest runtime, highest accuracy, and lowest memory usage of the three algorithms tested, making it the best suited algorithm for identifying Higgs boson events. Three varying BDT algorithms were tested throughout this study, the best performing was the histogram gradient boosted decision tree. The gradient boosted decision tree had a low accuracy and high training time without hyperparameter tuning. The random forest classifier was more comparable to the histogram gradient boosted tree. However, the training time of the random forest averaged around one minute and 30 seconds whereas the histogram gradient boosted decision tree was almost 11 times faster, averaging around 8.4 seconds to train. The decision tree also had the shortest runtime in comparison to the SVM and neural network by a considerable margin. While the decision tree required less than 10 seconds, the SVM took 50 minutes and the neural network required over an hour and a half. Similarly, the boosted decision tree required only 645 MB of memory while the neural network required 1029 MB and the SVM required 1168 MB.

The SVM was the worst in terms of accuracy but had a better training time at 50 minutes and 32 seconds than the neural network which took an hour and 34 minutes and 31 seconds to train (Table 1). This might suggest that the SVM is better suited for practical work as it does not require as much time to train. However, the SVM did require more memory usage than the neural network in this case, a factor which may detract from its ability to be practically implemented.

4.2. Comparison to Other Algorithms

On average, these algorithms performed at a capacity comparable to algorithms which are currently employed at CERN. The histogram gradient boosted decision tree had a higher accuracy and a lower training time than that of (Chen & He, 2014) which was within the top 2% performing algorithms of over 1,700 in the ATLAS Higgs ML Challenge from 2014 (Chen & He, 2014). The histogram gradient boosted decision tree also had a slightly worse test accuracy than the best performing BDT algorithm in the same challenge. The top accuracy for that algorithm was 84% (Ahmad, 2015) while the algorithm from this study averaged at an accuracy of 83.88%.

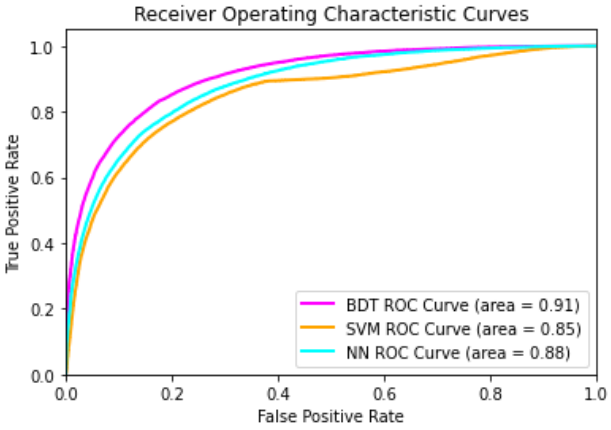


Figure 10. Receiver operating characteristic (ROC) curve for the three algorithms tested. A larger area under the curve (AUC) correlates with a better performance; the BDT has the highest AUC at 0.91, thus indicating that it is the most accurate model of the three tested.

The SVM performed much better than those in the challenge. Those with RBF kernels, as were used in this model, were incapable of performing and those with linear kernels had an accuracy of 76%, which is 4% lower than the accuracy achieved with this model (Ahmad, 2015).

The results of this study concur with the current implementation of algorithms at CERN for the classification of Higgs boson events from the background, in that GBDTs are the most effective tool for sorting data in offline reconstruction (Guest et al., 2018).

5. Conclusion

In this paper, three supervised classification algorithms were explored regarding their ability to classify Higgs boson events from the background using simulated data. Concurring with previously created algorithms and the current commonly used algorithm at CERN, the BDT had the best performance of all three algorithms in terms of accuracy of identification, runtime, and memory usage. The use of a histogram gradient boosted decision did help to further reduce training time from that of a regular gradient boosted decision tree as it binned data and was able to sort through it much more quickly while maintaining a high rate of accuracy. Though the SVM and neural network did not perform as well as the histogram gradient boosted decision tree, they were capable of performing at comparable or higher ac-

	Histrogram GBDT	SVM	Neural Network
Area Under the Curve	91%	84%	88%
Average Test Accuracy	83.88%	80.35%	82.48%
Average Test F1-Score	81.72%	77.12%	80.01%
Training Time (h:min:s)	00:00:8.4	00:50:32	1:34:31
Memory Usage (MB)	645.41	1167.60	1028.96

Table 1. Compares various features of each algorithm against each other, including average training set accuracy and F1-score, average testing set accuracy and F1-score, the time it took to train the algorithm, and the memory usage required to train and predict a dataset. The values reflecting the best performance for each category are highlighted in yellow while the worst performances are highlighted in red/pink.

curacies and lower training times than previously existing algorithms of similar types in the field. These algorithms can be practically implemented at CERN's offline reconstruction as means of sorting through candidate Higgs boson event data in order to identify Higgs bosons. There also exists a wide range of alternative work involving machine learning and its application to particle physics. Machine learning might be used in order to help sort through candidate dark matter events in order to aid in the search for dark matter or to identify tau leptons. It may be worthwhile to conduct a project in the future which looks at machine learning applications to other particles which are being sought after. Additionally, deep learning algorithms are in use to reconstruct Higgs boson events for further study. Improving current machine learning models and their applications to particle physics may provide an edge to confirming the SM and understanding our universe at a fundamental level.

References

Aad, G., Abajyan, T., Abbott, B., Abdallah, J., Abdel Khalek, S., Abdelalim, A., Abdinov, O., Aben, R., Abi, B., Abolins, M., and et al. Observation of a new particle in the search for the standard model higgs boson with the atlas detector at the lhc. *Physics Letters B*, 716(1):1–29, Sep 2012. ISSN 0370-2693. doi: 10.1016/j.physletb.2012.08.020.

Adam-Bourdarios, C., Cowan, G., Germain, C., Guyon, I., Kégl, B., and Rousseau, D. The Higgs boson machine learning challenge. In Cowan, G., Germain, C., Guyon, I., Kégl, B., and Rousseau, D. (eds.), *Proceedings of the NIPS 2014 Workshop on High-energy Physics and Machine Learning*, volume 42 of *Proceedings of Machine Learning Research*, pp. 19–55, Montreal, Canada, 13 Dec 2015. PMLR.

Ahmad, S. R. Technical report of participation in higgs boson machine learning challenge, 2015.

and, A. C. Review of standard model higgs results at the ATLAS experiment. *Journal of Physics: Conference Series*, 873:012003, jul 2017. doi: 10.1088/1742-6596/873/1/012003.

Bianchi, R. M. Higgs candidate decaying to 2 tau leptons in the ATLAS detector. Nov 2013. General Photo.

Chatrchyan, S., Khachatryan, V., Sirunyan, A., Tumasyan, A., Adam, W., Aguilo, E., Bergauer, T., Dragicevic, M., Erö, J., Fabjan, C., and et al. Observation of a new boson at a mass of 125 gev with the cms experiment at the lhc. *Physics Letters B*, 716(1):30–61, Sep 2012. ISSN 0370-2693. doi: 10.1016/j.physletb.2012.08.021.

Chen, T. and He, T. Higgs boson discovery with boosted trees. In *Proceedings of the 2014 International Conference on High-Energy Physics and Machine Learning - Volume 42*, HEPML'14, pp. 69–80. JMLR.org, 2014.

Englert, F. and Brout, R. Broken symmetry and the mass of gauge vector mesons. *Phys. Rev. Lett.*, 13:321–323, Aug 1964. doi: 10.1103/PhysRevLett.13.321.

Gritsan, A., Röntsch, R., Schulze, M., and Xiao, M. Constraining anomalous higgs boson couplings to the heavy favor fermions using matrix element techniques. *Physical Review D*, 94, 06 2016. doi: 10.1103/PhysRevD.94.055023.

Guest, D., Cranmer, K., and Whiteson, D. Deep learning and its application to lhc physics. *Annual Review of Nuclear and Particle Science*, 68(1):161–181, 2018. doi: 10.1146/annurev-nucl-101917-021019.

Higgs, P. W. Broken symmetries and the masses of gauge bosons. *Phys. Rev. Lett.*, 13:508–509, Oct 1964. doi: 10.1103/PhysRevLett.13.508.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Sen, P. C., Hajra, M., and Ghosh, M. Supervised classification algorithms in machine learning: A survey and review. In Mandal, J. K. and Bhattacharya, D. (eds.), *Emerging Technology in Modelling and Graphics*, pp. 99–111, Singapore, 2020. Springer Singapore. ISBN 978-981-13-7403-6.

Sforza, F. and Lippi, V. Support vector machine classification on a biased training set: Multi-jet background rejection at hadron colliders. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 722: 11–19, 2013. ISSN 0168-9002. doi: https://doi.org/10.1016/j.nima.2013.04.046.